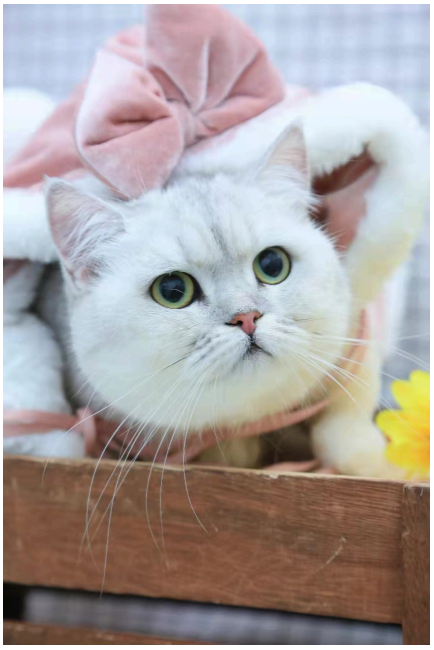
calc_coupling

Calculates charge transfer integrals using 2-states model with frontier orbitals approximation method using Gaussian output files.



A brief introduction of the algorithm

We use $\boldsymbol{F}_{\text{dimer}}$ as the Fock matrix of dimer, $\boldsymbol{S}_{\text{dimer}}$ as the basis functions overlap matrix of dimer, $\boldsymbol{C}_{\text{monomer1}}$ and $\boldsymbol{C}_{\text{monomer2}}$ as orbitals coefficient matrixes on the basis of monomer basis functions of monomer 1 and 2 separately. Define combined fragments molecule orbitals coefficient matrix $\boldsymbol{C}_{\text{frags}}$ as:

$$\boldsymbol{C}_{\text{frags}} \equiv \begin{bmatrix} \boldsymbol{C}_{\text{monomer1}} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{C}_{\text{monomer2}} \end{bmatrix}.$$

Then the interaction between two monomers at the environment of dimer is:

$$\boldsymbol{F}_{\text{frags}} = \boldsymbol{C}_{\text{frags}}^{\top} \boldsymbol{F}_{\text{dimer}} \boldsymbol{C}_{\text{frags}},$$

a.k.a.

$$\boldsymbol{F}_{\text{frags}} = \left\langle \boldsymbol{C}_{\text{frags}} \middle| \hat{\boldsymbol{F}}_{\text{dimer}} \middle| \boldsymbol{C}_{\text{frags}} \right\rangle.$$

The nonorthogonality of monomers orbitals is:

$$\boldsymbol{S}_{\text{frags}} = \boldsymbol{C}_{\text{frags}}^{\top} \boldsymbol{S}_{\text{dimer}} \boldsymbol{C}_{\text{frags}},$$

a.k.a.

$$\boldsymbol{S}_{\text{frags}} = \left\langle \boldsymbol{C}_{\text{frags}} \middle| \hat{\boldsymbol{S}}_{\text{dimer}} \middle| \boldsymbol{C}_{\text{frags}} \right\rangle.$$

For frontier orbitals approximation, extra electron transfers from monomer 1 to monomer 2 via their LUMOs, extra hole transfers from monomer 1 to monomer 2 via their HOMOs. For orbitals approximation model, let i and j be indices of initial orbital of monomer 1 (e.g. LUMO of it) and final orbital of monomer 2 (e.g. LUMO of it) in $\boldsymbol{C}_{\text{frags}}$ where the carrier is transferred from and to, then the Hamiltonian of this 2-states model is:

$$\boldsymbol{F} = \begin{bmatrix} e_i & J_{ij} \\ J_{ij} & e_j \end{bmatrix},$$

where $J_{ij} = \boldsymbol{F}_{\text{frags}_{i,j}}$, $e_i = J_{ii}$, $e_j = J_{jj}$. the J_{ij} is the transfer integral between orbital i and j at **nonorthogonal basis**. The nonorthogonality is caused by the 2-states overlap matrix:

$$\boldsymbol{S} = \begin{bmatrix} 1 & S_{ij} \\ S_{ij} & 1 \end{bmatrix} \neq \boldsymbol{I},$$

where $S_{ij} = \boldsymbol{S}_{\text{frags}_{i,j}}$ is generally not zero. To find out the **effective** transfer integral (a.k.a. coupling) between orbital i and j at **orthogonal** basis, a Löwdin orthogonalization is needed, that is so transform from a generalized eigen problem $\boldsymbol{FC} = \boldsymbol{SC\varepsilon}$ to a simple eigen problem $\boldsymbol{F'C'} = \boldsymbol{C'\varepsilon}$. To do this, for positive-defined matrix $\boldsymbol{S}$, solving its eigenvalues and eigenvectors by diagonalization gives:

$$\boldsymbol{S} \equiv \boldsymbol{Us}\boldsymbol{U}^\top,$$

where $\boldsymbol{s}$ is a diagonal matrix with all off-diagonal elements being zero, $\boldsymbol{U}$ is a unitary matrix such that $\boldsymbol{U}^\top\boldsymbol{U} = \boldsymbol{I}$. Define $\boldsymbol{s}^{\frac{1}{2}}$ as a diagonal matrix whose diagonal elements are the square roots of those corresponding to $\boldsymbol{s}$ (since $\boldsymbol{s}$ is positive-defined we can do this to make $\boldsymbol{s}^{\frac{1}{2}}$ real), and $s^{-\frac{1}{2}}$ as another diagonal matrix whose diagonal elements are the reciprocal of those corresponding to $\boldsymbol{s}^{\frac{1}{2}}$, we have:

$$\boldsymbol{S} = \boldsymbol{Us}^{\frac{1}{2}}\boldsymbol{s}^{\frac{1}{2}}\boldsymbol{U}^\top = \left(\boldsymbol{Us}^{\frac{1}{2}}\boldsymbol{U}^\top\right)\left(\boldsymbol{Us}^{\frac{1}{2}}\boldsymbol{U}^\top\right),$$

let $\boldsymbol{S}^{\frac{1}{2}} \equiv \boldsymbol{Us}^{\frac{1}{2}}\boldsymbol{U}^\top$, its inverse matrix $\boldsymbol{S}^{-\frac{1}{2}}$ will be $\boldsymbol{Us}^{-\frac{1}{2}}\boldsymbol{U}^\top$. Left-multiply $\boldsymbol{S}^{-\frac{1}{2}}$ to both sides of equation $\boldsymbol{FC} = \boldsymbol{SC\varepsilon}$ gives:

$$\begin{aligned} &\left(\boldsymbol{S}^{-\frac{1}{2}}F\boldsymbol{S}^{-\frac{1}{2}}\right)\left(\boldsymbol{S}^{\frac{1}{2}}\boldsymbol{C}\right) \\ =&\boldsymbol{S}^{-\frac{1}{2}}\boldsymbol{FC} \\ =&\boldsymbol{S}^{-\frac{1}{2}}\boldsymbol{SC\varepsilon} \\ =&\boldsymbol{S}^{-\frac{1}{2}}\boldsymbol{S}^{\frac{1}{2}}\boldsymbol{S}^{\frac{1}{2}}\boldsymbol{C\varepsilon} \\ =&\left(\boldsymbol{S}^{\frac{1}{2}}\boldsymbol{C}\right)\boldsymbol{\varepsilon} \end{aligned} .$$

Let

$$\begin{bmatrix} {e_i}^{\text{eff}} & {J_{ij}}^{\text{eff}} \\ {J_{ij}}^{\text{eff}} & {e_j}^{\text{eff}} \end{bmatrix} \equiv \boldsymbol{F'} = \boldsymbol{S}^{-\frac{1}{2}}F\boldsymbol{S}^{-\frac{1}{2}},$$

$$\boldsymbol{C'} = \boldsymbol{S}^{\frac{1}{2}}\boldsymbol{C},$$

(hence $\boldsymbol{C} = \boldsymbol{S}^{-\frac{1}{2}}\boldsymbol{C'}$). $\boldsymbol{F'}$ is the effective Hamiltonian of this 2-states model and ${J_{ij}}^{\text{eff}}$ is the **effective coupling** between orbitals i and j. Analytically solving Löwdin orthogonalization of this simple 2-states model gives:

$$
\begin{cases}
{e_i}^{\text{eff}} = \frac{1}{2}\frac{(e_i+e_j)-2F_{ij}S_{ij}+(e_i-e_j)\sqrt{1-{S_{ij}}^2}}{1-{S_{ij}}^2} \\
{e_j}^{\text{eff}} = \frac{1}{2}\frac{(e_i+e_j)-2F_{ij}S_{ij}+(e_j-e_i)\sqrt{1-{S_{ij}}^2}}{1-{S_{ij}}^2} \\
{J_{12}}^{\text{eff}} = \frac{F_{ij}-\frac{1}{2}(e_i+e_j)S_{ij}}{1-{S_{ij}}^2}
\end{cases}.
$$

For details see [10.1021/ja061827h](10.1021/ja061827h).

Application

Only **Gaussian** output file and formatted checkpoint files are supported. Only single point energy task is supported.

Phase-matching method of orbitals is currently **NOT** implied currently, hence **ONLY** the absolute value of the transfer integral calculated by this program is currently meaningful.

Please read the usage below carefully.

Compilation

This executable `calc_coupling` (or `calc_coupling.exe` on Windows) is compiled by **gfortran 10.3.0 x64** from TDM-gcc on Windows with **OpenBLAS 0.3.30** compiled with the same compilers and **cmake 3.25.2 x64** for Windows, and **ifort** and **MKL** within Intel OneAPI 2021 with **cmake 3.20.2** on Linux. If you want to compile yourself, you need to provide your own Linear Algebra libraries including **Intel MKL**, **OpenBLAS** or **NetLib LAPACK+BLAS**. The later two is suggested to be compiled by CMake on both Windows and Linux, while the former can be installed directly by following the official website of **Intel OneAPI**. Both **GNU gcc-gfortran** and **Intel ifort** are supported. A compiler that supports the Fortran 2008 standard is required. There is also a **C++** version of this code with **Eigen** and **fmt** libraries, you can acquire it by email or GitHub.

Usage

You can assign files names for dimer and monomers through command arguments. E.g. `calc_coupling dimer monomer1 monomer2` will read from `dimer.fch`, `monomer1.fch`, `monomer2.fch` and `dimer.out`. If `.fch` files are not found, `.fchk` files will be tried instead. If `.out` file is not found, `.log` file will be tried instead. Without command arguments, the default prefixes are as mentioned above.

The atom orders in monomer 1 should be the first part of dimer, and the orders in monomer 2 should be the second part of dimer.

`IOp(3/32=2) NoSymmetry` **MUST** present in **all** input files. `IOp(3/33=1)` **MUST** present in **dimer** input `.gjf` file. You can use `--Link1--` to provide an additional step for **dimer** to provide a Fock matrix with `IOp(5/33=3) Guess=Read NoSymmetry`. E.g.

```
%chk=dimer.chk
#P B3LYP/6-31G** IOp(3/32=2,3/33=1) NoSymmetry

Title Card Required

0 1
[GEOMETRY]

--Link1--
%chk=dimer.chk
#P B3LYP/ChkBasis IOp(5/33=3) NoSymmetry Geom=AllCheck Guess=Read
```

. If Fock matrix of **dimer** is not provided in the output file of Gaussian, it will be solved by orbitals coefficient matrix, orbitals energy vector and basis function overlap matrix of dimer by this program.

When the matrix of coefficients is singular, you can provide a plain text file containing the Fock Matrix of the Dimer. This file should have the first line as comment, followed by the lower-triangle format Fock matrix for dimer. The "lower-triangle format" can be found in examples\dimer_Fock_Gaussian.txt. For Gaussian users, add a Link-1 task after the input file for dimer with `Guess=Read IOp(5/33=3)` and the same checkpoint path as above. Then search for the last "Fock matrix (alpha)" or "Fock matrix (beta)" from the Gaussian output file and save it to a new plain text file. Then rerun this program with command argument `-dF your_file_name_for_Fock_matrix_for_dimer`.

Contact

For bug report and further discussion, please send emails to snljty@sina.com .